

# AI Code Completion Model

## Complete Semester Project Documentation

**Track:** Generative AI / Machine Learning

**Domain:** Software Development / Developer Tools

**Duration:** 12 weeks

**Level:** Intermediate

**Primary Language:** Python

**Base Model:** StarCoder2-3B

**Fine-tuning:** LoRA / PEFT

**Backend:** FastAPI

**Frontend:** React or Streamlit

**Note:** This document is adapted to the structure and level of detail of the sample project document provided, but the technical content is specific to the AI Code Completion Model project.

# Table of Contents

1. Project Overview
2. Project Description
3. Executive Summary
4. Problem Statement
5. Vision
6. Objectives
7. Target Users
8. Core Product Flow
9. MVP
10. Out of Scope
11. Business Requirements Document (BRD)
12. Product Requirements Document (PRD)
13. UX Requirements
14. Technical Requirements Document (TRD)
15. High-Level Design (HLD)
16. ML/Data Design
17. API Specification
18. Low-Level Design (LLD)
19. AI/ML Architecture
20. Security Design
21. Testing Strategy
22. CI/CD
23. Observability
24. Deployment Architecture
25. Cost Analysis
26. 12-Week Roadmap
27. Team Responsibilities
28. GitHub Repository Structure
29. Architecture Decision Records
30. Research Question & Hypothesis
31. Experimental Plan
32. Evaluation Plan
33. Viva / Project Defense
34. Definition of Done

# 1. Project Overview

| Item             | Details                                               |
|------------------|-------------------------------------------------------|
| Project Title    | AI Code Completion Model                              |
| Track            | Generative AI / Machine Learning                      |
| Domain           | Software Development / Developer Tools                |
| Duration         | 12 weeks                                              |
| Team             | 2 students                                            |
| Skill Level      | Intermediate                                          |
| Primary Users    | Software developers, students, programmers            |
| Primary Language | Python                                                |
| Model            | StarCoder2-3B                                         |
| Fine-tuning      | LoRA / PEFT                                           |
| Backend          | Python + FastAPI                                      |
| Frontend         | React or Streamlit                                    |
| ML Stack         | PyTorch + Hugging Face Transformers + Datasets + PEFT |
| Constraint       | GPU availability                                      |

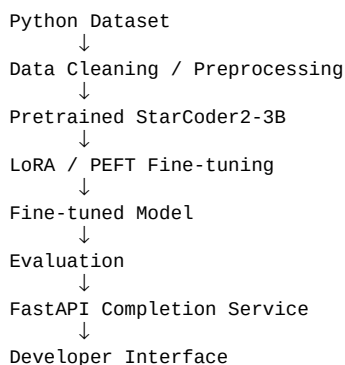
## 2. Project Description

The AI Code Completion Model is an AI-powered developer assistance system that predicts and generates the next portion of source code based on the code already written by a developer. The system uses a pretrained code language model and fine-tunes it on a curated Python source-code dataset using parameter-efficient fine-tuning techniques such as LoRA.

The final system exposes code completion through a simple developer-facing interface. The project focuses on model adaptation, measurable evaluation, and practical deployment rather than training a large language model from scratch.

## 3. Executive Summary

The project investigates whether fine-tuning a pretrained code language model on a curated Python dataset improves code-completion performance compared with the original pretrained model.



## 4. Problem Statement

Software developers frequently write repetitive code and repeatedly implement common programming patterns such as loops, functions, conditionals, API calls, data-processing operations, classes, and error handling.

Traditional autocomplete systems often rely on syntax rules, static analysis, symbols, and previously known identifiers. Modern code language models can use broader programming context to predict more sophisticated continuations.

Large code models, however, can require substantial computational resources. This project therefore investigates whether a smaller pretrained code model can be adapted using efficient fine-tuning to provide useful Python code completion under realistic student-project constraints.

## 5. Vision

Build a lightweight AI-powered code-completion system that understands programming context, predicts likely code continuations, produces useful suggestions, can be evaluated quantitatively, and operates with manageable computational resources.

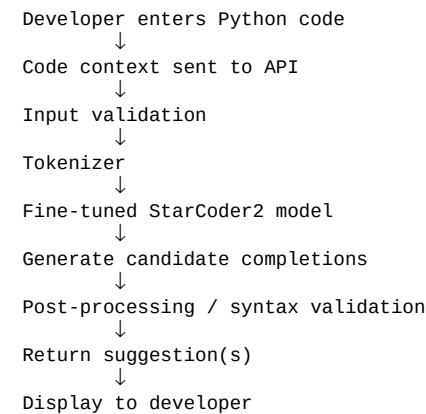
## 6. Objectives

- Study transformer-based code language models.
- Use a pretrained code language model rather than training from scratch.
- Prepare a clean Python programming dataset.
- Fine-tune the model using LoRA/PEFT.
- Build a code-completion inference service.
- Compare pretrained and fine-tuned models.
- Measure exact match, Top-k accuracy, perplexity, syntax validity, and latency.
- Build a simple developer-facing interface.
- Document limitations, errors, and computational requirements.

## 7. Target Users

| User                      | Goals                            | Pain Points                |
|---------------------------|----------------------------------|----------------------------|
| Software Developer        | Write code faster                | Repetitive coding          |
| Student Programmer        | Get contextual coding assistance | Syntax and API recall      |
| ML/AI Student             | Experiment with code models      | Complex model setup        |
| Developer Tool Researcher | Evaluate code completion         | Need measurable benchmarks |

## 8. Core Product Flow



## 9. MVP

- Python dataset collection and cleaning.
- Training/validation/test split.

- Pretrained model loading and baseline inference.
- LoRA fine-tuning.
- Fine-tuned model inference.
- Code-completion API.
- Simple code editor interface.
- Evaluation pipeline.
- Baseline versus fine-tuned comparison.
- Basic inference and experiment logging.

## 10. Out of Scope

- Training a large language model from scratch.
- Building a Copilot-scale IDE product.
- Autonomous software development.
- Automatic execution of arbitrary generated code.
- Automatic deployment of generated code.
- Equal support for every programming language in the MVP.
- Production-scale inference infrastructure.
- Use of private source code without authorization.

## 11. Business Requirements Document (BRD)

### 11.1 Business Objective

Develop a prototype AI developer-assistance system capable of generating useful Python code completions while demonstrating both machine-learning methodology and software-engineering practices.

### 11.2 Business Requirements

| ID     | Requirement                                         | Priority | Acceptance Criteria                    |
|--------|-----------------------------------------------------|----------|----------------------------------------|
| BR-001 | System shall accept Python code context             | Must     | Valid Python context is accepted       |
| BR-002 | System shall generate code completion               | Must     | Completion is returned                 |
| BR-003 | System shall support pretrained model inference     | Must     | Base model produces output             |
| BR-004 | System shall support fine-tuned model inference     | Must     | Fine-tuned model produces output       |
| BR-005 | System shall use a curated dataset                  | Must     | Dataset preprocessing pipeline exists  |
| BR-006 | System shall evaluate model performance             | Must     | Evaluation metrics are generated       |
| BR-007 | System shall compare baseline and fine-tuned models | Must     | Comparison report exists               |
| BR-008 | System shall provide a user interface               | Must     | User can enter code and see completion |
| BR-009 | System shall validate generated Python syntax       | Should   | Invalid suggestions are flagged        |
| BR-010 | System shall record model version                   | Should   | Model/version appears in metadata      |
| BR-011 | System shall support multiple candidates            | Could    | Top-k suggestions can be displayed     |
| BR-012 | System shall measure inference latency              | Should   | Response time is recorded              |

### 11.3 Non-Functional Requirements

- Performance: interactive completion should target an acceptable response time; latency must be measured.
- Reliability: model loading and inference failures must be handled gracefully.

- Security: generated code must not be automatically executed.
- Maintainability: training, inference, API, and UI components should be separated.
- Reproducibility: record model, dataset, training configuration, seed, and evaluation configuration.
- Scalability: keep the architecture modular so a stronger inference server can be introduced later.

## 11.4 Success Metrics

- Exact Match
- Top-1 / Top-3 / Top-5 accuracy
- Perplexity
- Python syntax validity
- Inference latency
- Model memory usage
- Completion usefulness / acceptance rate where measurable
- Improvement over pretrained baseline

## 11.5 Business Risks

| Risk                                     | Probability | Impact | Mitigation                                                      |
|------------------------------------------|-------------|--------|-----------------------------------------------------------------|
| Insufficient GPU memory                  | Medium      | High   | Use LoRA, mixed precision, smaller batches, cloud GPU           |
| Low-quality dataset                      | Medium      | High   | Cleaning, deduplication, repository-level split                 |
| Data leakage                             | Medium      | High   | Project/repository-level train-test separation                  |
| Poor generated code                      | Medium      | High   | Evaluation, syntax checks, human review                         |
| Slow inference                           | Medium      | Medium | Quantization/efficient settings and hardware profiling          |
| Model bias toward common coding patterns | Medium      | Medium | Diverse dataset and error analysis                              |
| License/compliance issues                | Low/Medium  | High   | Use datasets with appropriate licensing and document provenance |

# 12. Product Requirements Document (PRD)

## 12.1 Product Goal

Build an AI-powered Python code-completion system using a pretrained code language model fine-tuned with LoRA on curated Python source code.

## 12.2 Feature Priorities

| Feature                    | Priority |
|----------------------------|----------|
| Code input editor          | Must     |
| Base model inference       | Must     |
| Dataset preprocessing      | Must     |
| LoRA fine-tuning           | Must     |
| Fine-tuned model inference | Must     |
| Code completion            | Must     |
| Evaluation pipeline        | Must     |
| Baseline comparison        | Must     |
| Syntax validation          | Should   |
| Multiple suggestions       | Should   |

| Feature            | Priority |
|--------------------|----------|
| Completion history | Could    |
| VS Code extension  | Could    |

## 12.3 User Stories

### US-001

As a developer, I want to enter Python code so that the system can suggest the next part of the code.

### US-002

As a developer, I want the model to suggest a code continuation so that I can reduce repetitive coding.

### US-003

As a researcher, I want to compare the pretrained model with the fine-tuned model so that I can determine whether fine-tuning improves performance.

### US-004

As a researcher, I want to evaluate generated code so that I can measure model quality.

### US-005

As a developer, I want to see multiple possible suggestions so that I can select the most appropriate completion.

## 12.4 Product States

IDLE → INPUT\_RECEIVED → PROCESSING → GENERATING → COMPLETION\_READY → DISPLAYED  
 Failure: GENERATION\_FAILED  
 Model states: MODEL\_LOADING → MODEL\_READY / MODEL\_LOAD\_FAILED

## 12.5 Product Principles

- Context before completion.
- Useful suggestions over unnecessary suggestions.
- Human remains in control.
- Generated code is not executed automatically.
- Model limitations are visible.
- Evaluation is reproducible.
- AI output is treated as a suggestion, not guaranteed-correct code.

# 13. UX Requirements

## 13.1 Information Architecture

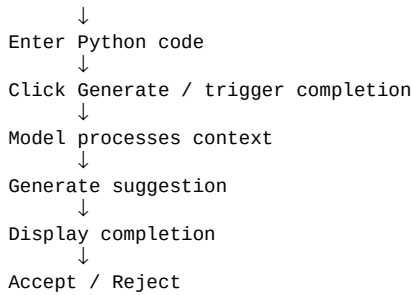
```

Home
↓
Code Completion Workspace
↓
Code Editor
↓
Completion Suggestions
↓
Model Information
↓
Evaluation / Metrics

```

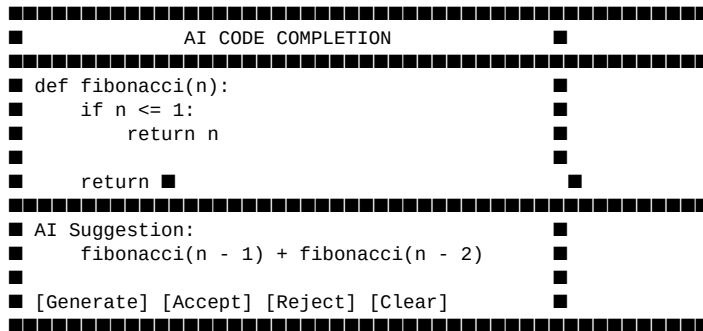
## 13.2 Primary User Flow

Open application



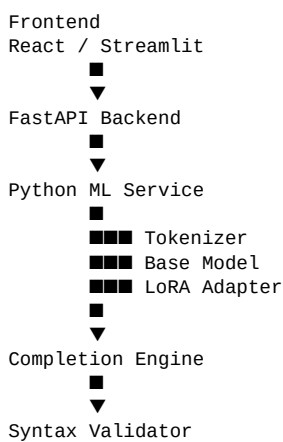
### 13.3 Main Screen Requirements

- Code editor with syntax highlighting.
- Generate completion action.
- Generated suggestion panel.
- Accept/reject controls.
- Model/version information.
- Latency display for demonstration.
- Error and loading states.



## 14. Technical Requirements Document (TRD)

### 14.1 Proposed Architecture



### 14.2 Technology Stack

| Technology                | Purpose                                 |
|---------------------------|-----------------------------------------|
| Python                    | Primary language and ML service         |
| PyTorch                   | Deep learning framework                 |
| Hugging Face Transformers | Model loading, tokenization, generation |

| Technology            | Purpose                                     |
|-----------------------|---------------------------------------------|
| Hugging Face Datasets | Dataset management                          |
| PEFT / LoRA           | Parameter-efficient fine-tuning             |
| StarCoder2-3B         | Pretrained code language model              |
| FastAPI               | Inference API                               |
| React                 | Developer-facing frontend (optional)        |
| Streamlit             | Simpler alternative frontend/prototyping UI |
| Git/GitHub            | Version control                             |

## 14.3 Technology Rationale

**StarCoder2-3B:** A pretrained code-oriented model that is more appropriate for this project than a general text model. Its size is more practical than very large code models for a student project.

**LoRA:** Reduces the number of trainable parameters and therefore lowers fine-tuning resource requirements.

**Python:** Provides a strong ecosystem for PyTorch, Transformers, datasets, PEFT, FastAPI, and evaluation.

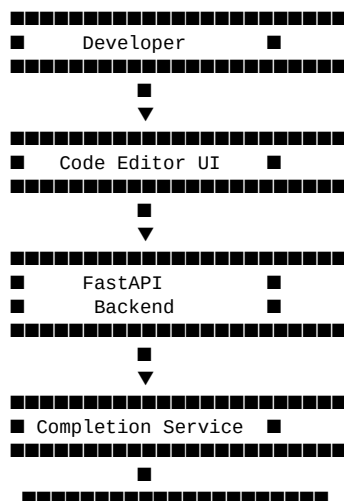
**FastAPI:** Lightweight API layer for model inference.

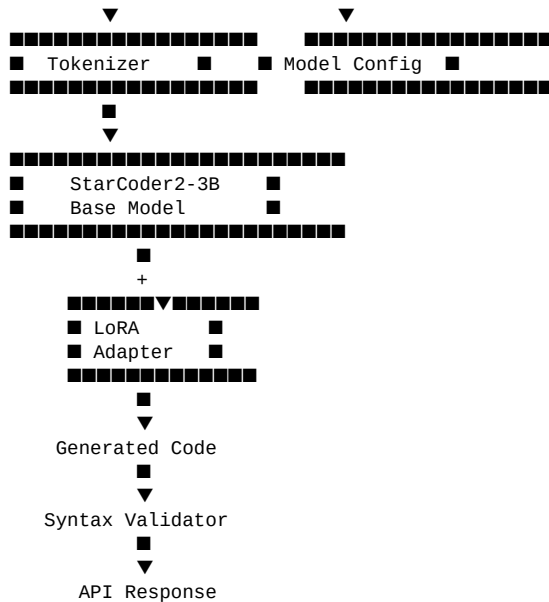
**React/Streamlit:** Provides either a realistic UI or a fast prototype interface.

## 14.4 Technical Requirements

| ID     | Requirement                      | Maps to        |
|--------|----------------------------------|----------------|
| TR-001 | Versioned REST API               | BR-001..BR-012 |
| TR-002 | Input validation                 | BR-001         |
| TR-003 | Pretrained model loading         | BR-003         |
| TR-004 | LoRA adapter loading             | BR-004         |
| TR-005 | Dataset preprocessing pipeline   | BR-005         |
| TR-006 | Reproducible evaluation pipeline | BR-006/007     |
| TR-007 | Syntax validation                | BR-009         |
| TR-008 | Model/version metadata           | BR-010         |
| TR-009 | Top-k generation                 | BR-011         |
| TR-010 | Latency measurement              | BR-012         |

## 15. High-Level Design (HLD)





## 15.1 Components

| Component          | Responsibility                            |
|--------------------|-------------------------------------------|
| React/Streamlit UI | Code editor and suggestion display        |
| FastAPI            | HTTP interface and request validation     |
| Inference Service  | Tokenization, generation, post-processing |
| StarCoder2-3B      | Base pretrained code language model       |
| LoRA Adapter       | Project-specific learned parameters       |
| Syntax Validator   | Checks generated Python syntax            |
| Evaluation Service | Computes model metrics                    |
| Experiment Store   | Stores training/evaluation metadata       |
| Logging            | Records safe operational metrics          |

## 15.2 End-to-End Data Flow

1. Developer enters code context.
2. Frontend sends completion request.
3. FastAPI validates request.
4. Tokenizer converts source code to tokens.
5. Model generates one or more candidate continuations.
6. Post-processing removes unwanted artifacts.
7. Python syntax validator checks generated code where applicable.
8. API returns suggestion(s), model version, and latency.
9. UI displays the completion.

## 15.3 Scalability

Start as a modular application rather than microservices. Keep the ML service separate from the UI so that GPU inference can later be moved to a dedicated server. Add queues or multiple workers only if workload requires them.

## 15.4 Reliability

- Model load failure handling.

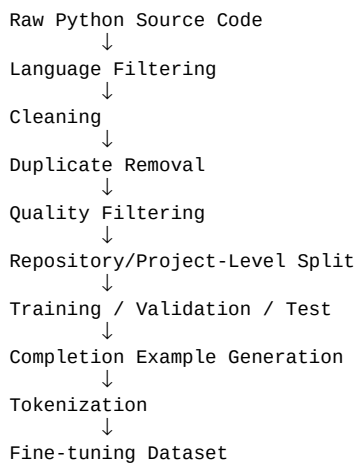
- Request timeout handling.
- Controlled generation limits.
- Graceful error responses.
- Health endpoint.
- Versioned model artifacts.
- Fallback to baseline model when appropriate, if implemented.

## 16. ML / Data Design

### 16.1 Dataset Objective

The dataset should contain source code suitable for training a Python code-completion model. The dataset pipeline must transform raw source code into context/continuation examples while reducing duplicates, corrupted samples, and leakage between train and test sets.

### 16.2 Dataset Pipeline



### 16.3 Dataset Split

| Split      | Recommended Share | Purpose                             |
|------------|-------------------|-------------------------------------|
| Training   | 80%               | Model adaptation                    |
| Validation | 10%               | Hyperparameter/checkpoint selection |
| Test       | 10%               | Final unseen evaluation             |

**Note:** Do not randomly split near-duplicate fragments from the same repository across train and test. Prefer repository/project-level separation to reduce data leakage.

### 16.4 Completion Example

```

Original:
def calculate_area(radius):
    pi = 3.14159
    area = pi * radius * radius
    return area

Context:
def calculate_area(radius):
    pi = 3.14159
    area =

Target:
pi * radius * radius
  
```

### 16.5 Data Quality Checks

- Remove empty files.

- Remove corrupted files.
- Limit extremely large files.
- Filter to Python for the MVP.
- Remove duplicates.
- Identify obvious generated/vendor files where possible.
- Scan for secrets where practical.
- Record dataset provenance and licensing information.

## 17. API Specification

Base URL: /api/v1

### 17.1 POST /completion

```
Request:
{
  "code": "def add(a, b):\n    return",
  "max_new_tokens": 32
}

Response:
{
  "completion": " a + b",
  "model": "starcoder2-3b",
  "modelVersion": "v1",
  "latencyMs": 245
}
```

### 17.2 POST /completion/top-k

```
Request:
{
  "code": "def add(a, b):\n    return",
  "k": 5
}

Response:
{
  "suggestions": [
    " a + b",
    " b + a",
    " sum([a, b])"
  ]
}
```

### 17.3 GET /model

```
{
  "model": "starcoder2-3b",
  "adapter": "code-completion-lora-v1",
  "version": "v1",
  "language": "python"
}
```

### 17.4 GET /health

```
{
  "status": "ok",
  "modelLoaded": true
}
```

### 17.5 Standard Errors

| Status | Meaning                     |
|--------|-----------------------------|
| 400    | Invalid request             |
| 404    | Endpoint/resource not found |
| 422    | Semantic validation failure |

| Status | Meaning                      |
|--------|------------------------------|
| 500    | Internal inference error     |
| 503    | Model/dependency unavailable |

## 18. Low-Level Design (LLD)

```

code-completion/
  frontend/
  backend/
  ai-service/
    model/
    inference/
    preprocessing/
    training/
    evaluation/
    schemas/
  data/
    raw/
    processed/
    test/
  models/
  experiments/
  tests/
  scripts/
  docs/

```

### 18.1 Key Interfaces

#### CodeCompletionModel

```

load()
generate(code_context)
generate_top_k(code_context, k)

```

#### DatasetProcessor

```

load_dataset()
clean()
deduplicate()
create_examples()
split()
tokenize()

```

#### Evaluator

```

exact_match()
top_k_accuracy()
perplexity()
syntax_validity()
latency()
generate_report()

```

### 18.2 Inference Sequence

```

Developer → Frontend: enter code
Frontend → FastAPI: POST /completion
FastAPI → Inference Service: validated context
Inference → Tokenizer: tokenize
Tokenizer → Model: token IDs
Model → Inference: generated token IDs
Inference → Validator: syntax check
Validator → FastAPI: completion result
FastAPI → Frontend: JSON response
Frontend → Developer: display suggestion

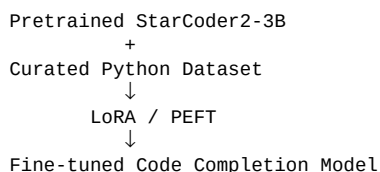
```

## 19. AI/ML Architecture

### 19.1 Why Generative AI?

Code completion is naturally modeled as conditional language generation: given a code context, predict a likely continuation. A transformer-based code language model can model relationships across a programming context and generate tokens sequentially.

## 19.2 Model Strategy



## 19.3 Next-Token Prediction

At a high level, the language model estimates the probability of the next token conditioned on the preceding context:  $P(\text{next token} \mid \text{previous tokens})$ . During generation, tokens are selected according to the configured decoding strategy.

## 19.4 Fine-Tuning Strategy

- Start with the pretrained model as a baseline.
- Attach LoRA adapters.
- Train only the parameter-efficient adapter components.
- Use validation data for checkpoint selection.
- Save the adapter and training configuration.
- Evaluate the resulting model on a fixed unseen test set.

## 19.5 Generation Strategies

| Strategy    | Description                             | Use                        |
|-------------|-----------------------------------------|----------------------------|
| Greedy      | Select highest-probability next token   | Simple baseline            |
| Top-k       | Sample/select from k likely tokens      | Multiple plausible outputs |
| Top-p       | Select from cumulative probability mass | Controlled diversity       |
| Temperature | Adjust distribution sharpness           | Control randomness         |

## 19.6 AI Limitations

- Generated code may be incorrect.
- Generated code may contain security vulnerabilities.
- The model can suggest outdated APIs.
- The model may produce syntactically invalid code.
- The model can repeat patterns from training data.
- Suggestions are not guarantees of correctness.

# 20. Security Design

## 20.1 Core Rules

- Never automatically execute model-generated code.
- Do not use `eval()` or `exec()` on arbitrary model output.
- Treat user code as untrusted input.
- Validate request size and schema.
- Keep secrets in environment variables or secret storage.
- Do not commit API keys.
- Avoid logging complete private source code.

## 20.2 API Security

- Input validation.
- Rate limiting where deployed publicly.
- CORS configuration.
- Secure headers.
- Authentication if required by deployment.
- Dependency scanning.

## 20.3 Model Security

- Protect model artifacts.
- Verify downloaded model sources.
- Track model versions.
- Restrict write access to model directories.
- Record dataset and model provenance.

# 21. Testing Strategy

Testing should cover software behavior and ML behavior. Model quality must be evaluated on data that was not used for training.

## 21.1 Test Levels

- Unit
- Integration
- API
- UI
- End-to-end
- Security
- Performance
- AI/ML evaluation

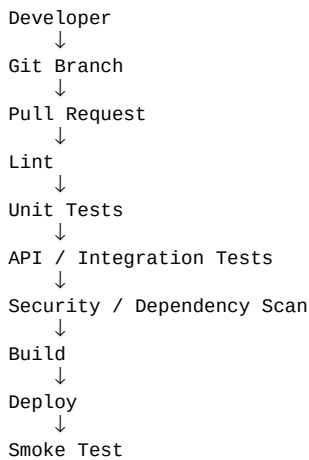
## 21.2 Sample Test Cases

| ID       | Scenario                 | Expected Result               | Priority |
|----------|--------------------------|-------------------------------|----------|
| TEST-001 | Valid Python input       | Completion generated          | High     |
| TEST-002 | Empty input              | Validation error              | High     |
| TEST-003 | Very large input         | Request limited/rejected      | High     |
| TEST-004 | Model unavailable        | Graceful error                | High     |
| TEST-005 | Valid completion         | Returned successfully         | High     |
| TEST-006 | Invalid generated syntax | Flagged                       | High     |
| TEST-007 | Top-k request            | Multiple suggestions returned | Medium   |
| TEST-008 | GPU unavailable          | Appropriate fallback/error    | High     |
| TEST-009 | API timeout              | Controlled error              | Medium   |
| TEST-010 | Model version request    | Correct version returned      | Medium   |

## 21.3 ML Evaluation

Use a fixed test set and compare the pretrained baseline with the fine-tuned model. Metrics should be selected based on the task and interpreted together rather than relying on a single score.

## 22. CI/CD



Model training should generally be separated from ordinary application CI. Training is an experiment pipeline with versioned data, configurations, checkpoints, and evaluation results.

## 23. Observability

### 23.1 Logs

- Request ID
- Model version
- Adapter version
- Processing state
- Error code
- Inference latency
- Input/output token counts

**Note:** Avoid logging complete source-code inputs unless there is a clear, authorized need.

### 23.2 Metrics

- API latency
- Inference latency
- Error rate
- Requests per minute
- GPU memory usage
- Token counts
- Syntax validity
- Completion acceptance rate if captured

### 23.3 Health

Expose a health endpoint that indicates whether the service is running and whether the model is loaded.

## 24. Deployment Architecture

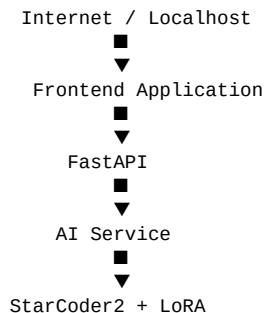
### 24.1 Development

Developer Machine  
■■■ Frontend  
■■■ FastAPI  
■■■ Model / AI Service

### 24.2 Training

Fine-tuning can be performed on a local NVIDIA GPU, university GPU server, or a cloud notebook/compute environment depending on available VRAM and project constraints.

### 24.3 Deployment



### 24.4 GPU Constraint

- Do not train from scratch.
- Use LoRA/PEFT.
- Use mixed precision where supported.
- Use gradient accumulation if necessary.
- Profile memory and latency.
- Use cloud GPU resources when local hardware is insufficient.
- Record hardware, training time, and configuration.

## 25. Cost Analysis

### 25.1 Cost Categories

- Model storage
- Dataset storage
- GPU training
- GPU inference
- Cloud hosting
- Network/storage if deployed

### 25.2 Measurement

Record:  
GPU type  
GPU VRAM  
Training time  
Dataset size  
Epochs  
Model/adaptor size  
Inference latency  
Number of inference requests

For a student project, the goal is to minimize compute cost while maintaining measurable improvement over the pretrained baseline.

## 26. 12-Week Project Roadmap

| Phase        | Weeks | Deliverables                                             |
|--------------|-------|----------------------------------------------------------|
| Research     | 1     | Literature review, code LLM concepts, problem definition |
| Requirements | 2     | BRD, PRD, UX                                             |
| Architecture | 3     | TRD, HLD, LLD                                            |
| Environment  | 4     | Python, PyTorch, Transformers setup                      |
| Baseline     | 5     | Pretrained model inference                               |
| Dataset      | 6     | Dataset cleaning and preprocessing                       |
| Fine-tuning  | 7     | LoRA training                                            |
| Evaluation   | 8     | Metrics and experiments                                  |
| Backend      | 9     | FastAPI completion service                               |
| Frontend     | 10    | Code editor interface                                    |
| Integration  | 11    | End-to-end integration and testing                       |
| Finalization | 12    | Deployment, documentation, presentation                  |

## 27. Team Responsibilities

### Student 1 — AI/ML

- Literature review
- Dataset collection
- Cleaning/preprocessing
- Model selection
- Baseline inference
- LoRA fine-tuning
- Evaluation
- Experiment tracking
- Model artifacts

### Student 2 — Application / Full Stack

- PRD/UX
- Frontend
- Code editor
- FastAPI integration
- API design
- Inference integration
- UI testing
- Deployment

### Shared

- Architecture
- Security
- Testing
- Deployment
- Documentation
- Presentation
- Viva preparation

## 28. GitHub Repository Structure

```

ai-code-completion/
├── docs/
│   ├── 01-project-overview.md
│   ├── 02-brd.md
│   ├── 03-prd.md
│   ├── 04-ux-requirements.md
│   ├── 05-trd.md
│   ├── 06-hld.md
│   ├── 07-ml-data-design.md
│   ├── 08-api-specification.md
│   ├── 09-lld.md
│   ├── 10-ai-architecture.md
│   ├── 11-security.md
│   ├── 12-testing.md
│   ├── 13-cicd.md
│   ├── 14-observability.md
│   ├── 15-deployment.md
│   ├── 16-cost-analysis.md
│   ├── 17-roadmap.md
│   ├── 18-team-responsibilities.md
│   ├── 19-experiments.md
│   └── 20-viva.md
├── frontend/
├── backend/
├── ai-service/
│   ├── training/
│   ├── inference/
│   ├── preprocessing/
│   ├── evaluation/
│   └── model/
├── data/
├── models/
├── experiments/
├── tests/
├── scripts/
├── requirements.txt
├── README.md
└── .gitignore

```

## 29. Architecture Decision Records (ADR)

### ADR-001 — Use Pretrained Model

**Decision:** Use StarCoder2-3B instead of training a model from scratch. **Rationale:** Training a large code model from scratch is outside the practical scope of a semester project. **Trade-off:** Dependence on an existing pretrained model.

### ADR-002 — Use LoRA

**Decision:** Use LoRA/PEFT. **Rationale:** Reduces trainable parameters and computational requirements. **Trade-off:** The adaptation is represented by an adapter rather than a fully retrained model.

### ADR-003 — Python First

**Decision:** Initially support Python. **Rationale:** Reduces complexity and allows focused evaluation. **Trade-off:** Other languages are outside the MVP.

## ADR-004 — Suggestions, Not Execution

**Decision:** Generated code is displayed as a suggestion. **Rationale:** AI-generated code can be incorrect or insecure. **Trade-off:** The developer must review the suggestion.

## 30. Research Question & Hypothesis

### 30.1 Research Question

Does parameter-efficient fine-tuning of a pretrained code language model improve Python code-completion performance compared with the original pretrained model?

### 30.2 Null Hypothesis (H0)

Fine-tuning does not produce a statistically meaningful improvement in Python code-completion performance.

### 30.3 Alternative Hypothesis (H1)

Fine-tuning on a curated Python dataset improves Python code-completion performance compared with the pretrained baseline.

## 31. Experimental Plan

### Experiment A — Baseline vs Fine-tuned

Model A: StarCoder2-3B, no fine-tuning  
Model B: StarCoder2-3B + LoRA adapter  
Compare both on the same unseen test set.

### Experiment B — Dataset Size

| Configuration                      | Purpose                |
|------------------------------------|------------------------|
| 10K examples                       | Small-data adaptation  |
| 50K examples                       | Medium-data adaptation |
| 100K examples, if resources permit | Larger-data adaptation |

### Experiment C — Generation Strategy

If time permits, compare greedy decoding with controlled Top-k/Top-p generation. Do not let this replace the primary baseline-versus-fine-tuned experiment.

## 32. Evaluation Plan

### 32.1 Exact Match

Determine whether the generated completion exactly matches the expected target under the defined evaluation normalization.

### 32.2 Top-k Accuracy

Measure whether the correct next token or completion candidate appears within the top k candidates, using a clearly defined evaluation protocol.

### 32.3 Perplexity

Measure the model's uncertainty over the test sequence. Lower perplexity generally indicates better next-token prediction.

### 32.4 Syntax Validity

Attempt to parse generated Python code using Python's AST/parser. Record the proportion of generated samples that are syntactically valid.

## 32.5 Latency

Measure time from request receipt to completion response and report mean/median/p95 where feasible.

## 32.6 Results Table

| Model         | Fine-tuning | Exact Match | Top-1 | Top-5 | Perplexity | Syntax Validity | Latency |
|---------------|-------------|-------------|-------|-------|------------|-----------------|---------|
| StarCoder2-3B | No          | TBD         | TBD   | TBD   | TBD        | TBD             | TBD     |
| StarCoder2-3B | LoRA        | TBD         | TBD   | TBD   | TBD        | TBD             | TBD     |

**Note:** Do not insert invented performance values. Populate this table only after running the experiments.

# 33. Viva / Project Defense

## 33.1 Beginner Questions

- What is code completion?
- What is an LLM?
- What is a transformer?
- What is self-attention?
- What is tokenization?
- What is StarCoder2?
- Why did you use a pretrained model?
- What is fine-tuning?
- What is LoRA?
- What is PyTorch?
- What is Hugging Face?
- What is inference?
- What is perplexity?
- What is Top-k accuracy?
- What is FastAPI?

## 33.2 Intermediate Questions

- Why didn't you train from scratch?
- Why StarCoder2-3B?
- Why Python?
- Why LoRA instead of full fine-tuning?
- How did you prepare the dataset?
- How did you prevent data leakage?
- How did you split the dataset?
- How did you evaluate the model?
- Why isn't accuracy alone sufficient?
- How do you test generated code?

- What happens if the model generates incorrect code?
- How does the tokenizer work?
- What is causal language modeling?
- How does next-token prediction work?
- What is the difference between training and inference?

### 33.3 Advanced Questions

- How does self-attention work?
- How does LoRA reduce trainable parameters?
- How does Fill-in-the-Middle differ from ordinary next-token completion?
- How would you support Java/C++/JavaScript?
- How would you reduce inference latency?
- How would you deploy the model at scale?
- How would you prevent insecure generated code?
- How would you detect dataset contamination?
- How would you compare two code language models fairly?

## 34. Definition of Done

- Project requirements documented.
- Dataset pipeline implemented and reproducible.
- Pretrained baseline tested.
- LoRA fine-tuning completed or experimentally validated.
- Fine-tuned model evaluated on unseen test data.
- Baseline versus fine-tuned comparison completed.
- FastAPI inference service implemented.
- Developer-facing interface implemented.
- Unit/API/integration tests added.
- Security rules documented and enforced.
- Model and experiment versions recorded.
- Deployment/demo environment prepared.
- Final report and presentation prepared.

## 35. Final Project Summary

The final system should be presented as an ML research and engineering project, not simply as a website around an existing model.

**Recommended project statement:** “We investigate parameter-efficient domain adaptation of a pretrained code language model for Python code completion, develop a curated training pipeline, fine-tune the model using LoRA, quantitatively compare it against the pretrained baseline, and integrate the resulting model into a developer-facing code-completion application.”

```

Python Dataset
  ↓
Cleaning / Deduplication
  ↓

```

